REPUBLIQUE DU CAMEROUN
********
PAIX-TRAVAL-PATRIE
********
UNIVERSITE DE DSCHAN
********
FACULTES DES SCIENCES
********
DEPARTEMENT DE SCIENCES

REPUBLIC OF CAMEROON
********
PEACE-WORK-FATHERLAND
********
UNIVERSITY OF DSCHANG
********
FACULTY OF SCIENCES
********
DEPARTEMENT OF SCIENCES

Architecture des Systèmes Intelligents

# Travaux Pratiques : Combinaison de classeurs, explicabilité des modèles

**Présenté par :**

| | |
|---|---|
| **DNJOMOU YONMBA WILFRIED LOIC** | **CM-UDS-24SCI0999** |
| **KENFACK LEONEL** | **CM-UDS-17SCI0998** |
| **SAGUEU WAKAM DILANE** | **CM-UDS-24SCI1040** |
| **NZOGNING MBONDA SOCRATE** | **CM-UDS-19SCI2910** |

**Superviseur : Pr. KENGNE TCHENDJI Vianney**

**Année académique 2025-2026**

# Table des matières


1 Etat de l'art . . . . . 4
2 Prétraitement et visualisation des données . . . . . 5
  2.1 Chargement et nettoyage . . . . . 5
  2.2 Gestion des valeurs manquantes et encodage . . . . . 5
  2.3 Visualisation des données . . . . . 6
  2.4 Découpage et équilibrage . . . . . 8
3 Interprétabilité intrinsèque des modèles et explicabilité post-hoc . . . . . 9
  3.1 Interprétabilité intrinsèque . . . . . 9
  3.2 Explicabilité post-hoc des modèles opaques . . . . . 11
  3.3 Synthèse . . . . . 14
4 Explicabilité des Modèles de Combinaison (Bagging et Boosting) . . . . . 15
5 Approches Avancées d'Explicabilité . . . . . 16

# Table des figures

2.1 Distributions des variables numériques du jeu CKD (après imputation). . . . . 6
2.2 Effectifs des variables catégorielles (avant encodage). . . . . 7
2.3 Matrice de corrélation (triangle inférieur) entre toutes les variables du jeu CKD. . 7
3.1 Coefficients de la régression logistique (effet sur la prédiction CKD). . . . . 10
3.2 Arbre de décision (max_depth=4) pour la prédiction CKD. . . . . 11
3.3 Poids LIME pour une observation de test : contribution de chaque variable à la probabilité CKD. . . . . 12
3.4 Valeurs SHAP (explication locale) pour une observation de test. . . . . 13
3.5 Résumé SHAP (vue globale) : impact des variables sur la prédiction CKD. . . . . 14

# Liste des tableaux

# 1 Etat de l'art

# 2 Prétraitement et visualisation des données

Cette section décrit la chaîne de prétraitement et d'exploration du jeu de données *Chronic Kidney Disease* (CKD) du dépôt UCI. L'objectif est de préparer les données pour les modèles d'explicabilité présentés aux sections suivantes : nettoyage, gestion des manquants, encodage, visualisation, puis découpage et équilibrage des classes. Toutes les étapes décrites correspondent à l'implémentation du notebook associé.

## 2.1 Chargement et nettoyage

Le jeu `kidney_disease.csv` contient 400 observations et 25 attributs (après suppression de l'identifiant). La cible est binaire : présence ou absence de maladie rénale chronique (CKD). La structure comprend :

— **Variables numériques** (14) : âge, tension artérielle (*blood_ pressure*), gravité spécifique (*specific_ gravity*), albumine, sucre, glucose sanguin, urée, créatinine sérique, sodium, potassium, hémoglobine, volume cellulaire, globules blancs/rouges, etc.

— **Variables catégorielles** (10) : globules rouges, cellules de pus, bactéries, hypertension, diabète, maladie coronarienne, appétit, œdème, anémie, etc.

Les colonnes ont été renommées de façon explicite (ex. `bp` → `blood_pressure`). Certaines colonnes initialement en format texte ont été converties en numérique (*packed_ cell_ volume*, *white_ blood_ cell_ count*, *red_ blood_ cell_ count*). Les valeurs catégorielles erronées (espaces, tabulations) ont été normalisées ; la cible a été encodée en binaire (0 = CKD, 1 = non CKD), et les lignes sans étiquette valide ont été supprimées (`dropna(subset=['class'])`).

```
df = pd.read_csv('kidney_disease.csv')
df.drop('id', axis=1, inplace=True)
df.columns = ['age', 'blood_pressure', 'specific_gravity', ...]
df['packed_cell_volume'] = pd.to_numeric(df['packed_cell_volume'],
    errors='coerce')
df['white_blood_cell_count'] = pd.to_numeric(df['white_blood_cell_count'
    ], errors='coerce')
df['red_blood_cell_count'] = pd.to_numeric(df['red_blood_cell_count'],
    errors='coerce')
df['class'] = df['class'].map({'ckd': 0, 'not␣ckd': 1})
df = df.dropna(subset=['class'])
```

## 2.2 Gestion des valeurs manquantes et encodage

Les valeurs manquantes ont été traitées de la manière suivante :

— **Variables numériques** : imputation par **KNN** (k plus proches voisins) avec $k = 5$ voisins (`KNNImputer` de scikit-learn), afin de préserver la structure locale des données.

— **Variables catégorielles** : remplacement des manquants par le **mode** de chaque colonne.

```
knn_imputer = KNNImputer(n_neighbors=5)
df[num_cols] = knn_imputer.fit_transform(df[num_cols])
for col in cat_cols:
    mode_val = df[col].mode()[0]
```

```
    df[col] = df[col].fillna(mode_val)
```

Ensuite, toutes les variables catégorielles ont été encodées numériquement à l'aide d'un **LabelEncoder** (une valeur entière par modalité), ce qui permet d'obtenir une matrice entièrement numérique pour la corrélation et les modèles.

```
le = LabelEncoder()
for col in cat_cols:
    df[col] = le.fit_transform(df[col].astype(str))
X = df.drop(columns=['class'])
y = df['class']
feature_names = X.columns.tolist()
```

## 2.3 Visualisation des données

La figure 2.1 présente les distributions des variables numériques (histogrammes). On observe des échelles et des formes variées (âge, pression, créatinine, hémoglobine, etc.), ce qui justifie l'utilisation d'un *StandardScaler* en aval pour les algorithmes sensibles à l'échelle.

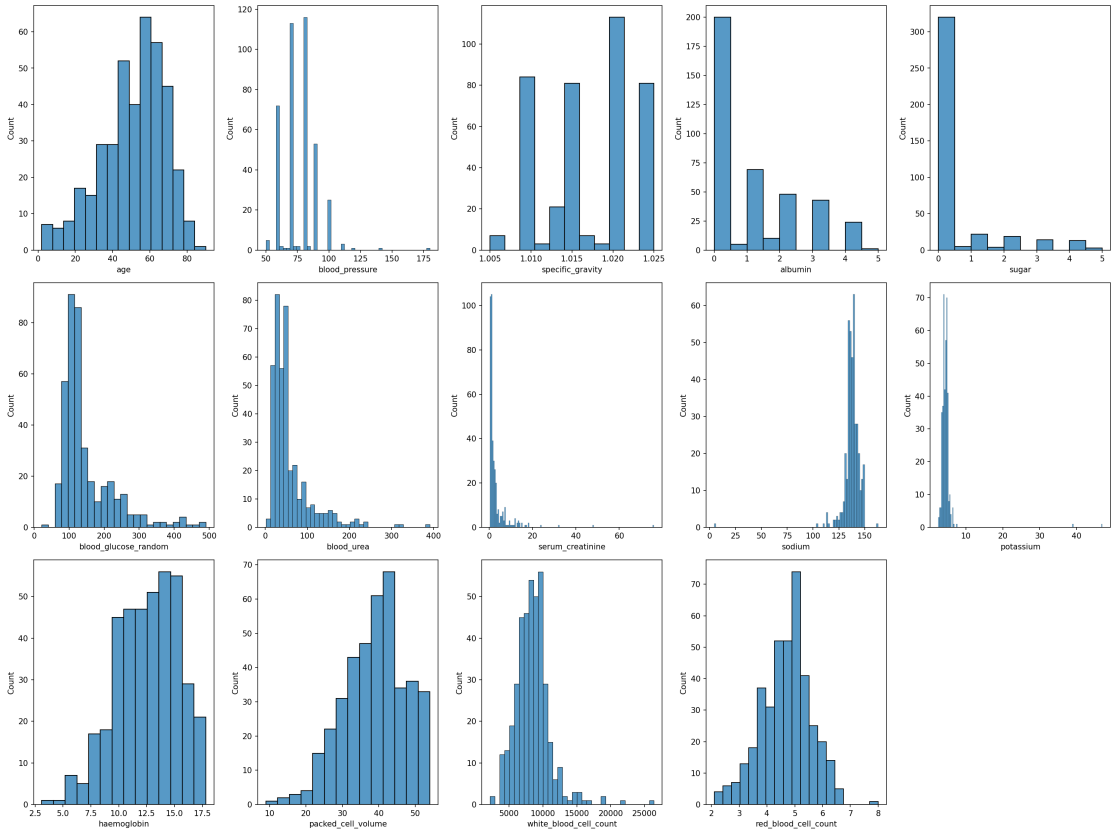


FIGURE 2.1 – Distributions des variables numériques du jeu CKD (après imputation).

La figure 2.2 montre les effectifs des modalités pour les variables catégorielles (avant encodage). Cela permet de vérifier l'équilibre relatif des catégories et d'identifier d'éventuelles modalités rares.

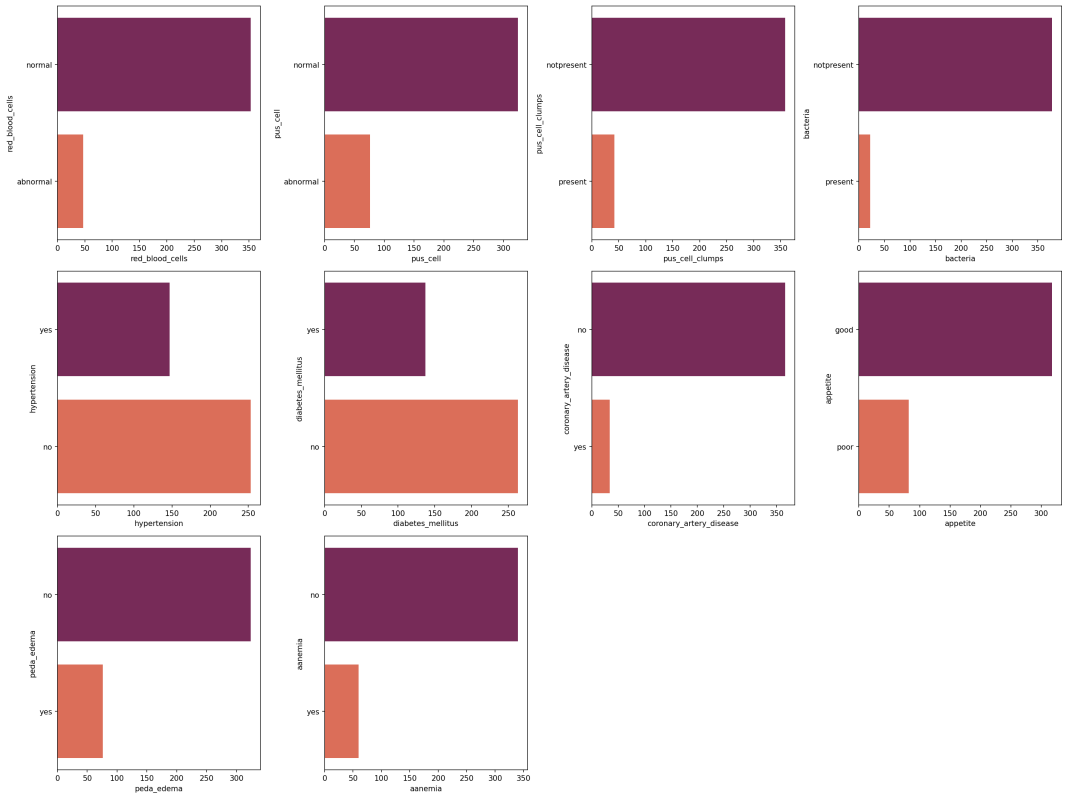


FIGURE 2.2 – Effectifs des variables catégorielles (avant encodage).

La figure 2.3 présente la matrice de corrélation. Les variables déjà encodées sont incluses ; on peut repérer les paires d'attributs fortement corrélés (ex. hémoglobine et volume cellulaire), utiles pour l'interprétation des modèles.

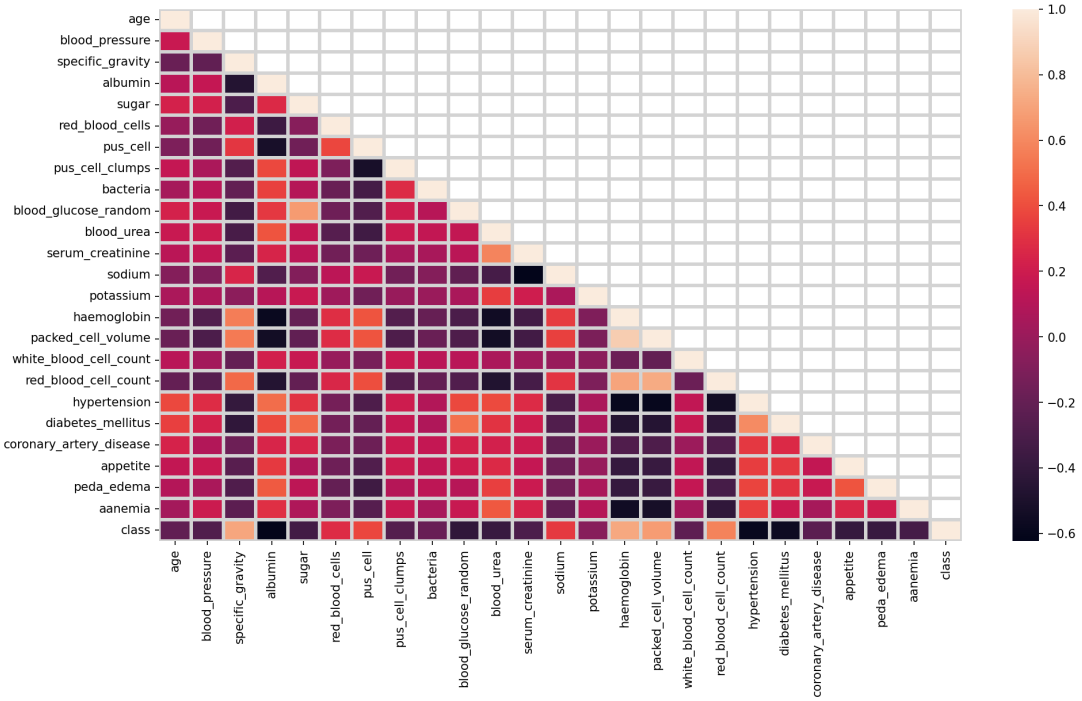


FIGURE 2.3 – Matrice de corrélation (triangle inférieur) entre toutes les variables du jeu CKD.

### 2.4 Découpage et équilibrage

Le jeu préparé est divisé en ensembles d'entraînement et de test par **découpage stratifié** (% 70 / 30) pour conserver les proportions de classes. La classe CKD étant souvent déséquilibrée, un rééquilibrage est appliqué sur l'ensemble d'entraînement uniquement avec **SMOTE** (technique de surotéchantillonnage de la minorité synthétique), avec $k = 3$ voisins, afin d'obtenir des effectifs de classes plus proches. Enfin, les variables numériques sont mises à l'échelle avec un **StandardScaler** (moyenne 0, écart-type 1), ajusté sur l'ensemble d'entraînement (après SMOTE) et appliqué aux données d'entraînement et de test. Les matrices ainsi obtenues (`X_train_scaled`, `X_test_scaled`) et les étiquettes associées sont utilisées dans les sections suivantes pour la régression logistique, l'arbre de décision, le SVM, LIME et SHAP.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=RANDOM_STATE)
smote = SMOTE(random_state=RANDOM_STATE, k_neighbors=3)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
scaler = StandardScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train_smote),
   columns=feature_names)
X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=
   feature_names, index=X_test.index)
```

# 3 Interprétabilité intrinsèque des modèles et explicabilité post-hoc

Les données utilisées sont celles préparées à la section précédente (prétraitement et visualisation). On vise à illustrer deux approches complémentaires de l'explicabilité sur le jeu CKD : (1) l'**interprétabilité intrinsèque** via des modèles dont la structure fournit directement des explications (régression logistique, arbre de décision) ; (2) l'**explicabilité post-hoc** en expliquant a posteriori un modèle « boîte noire » (SVM à noyau) à l'aide de LIME et SHAP.

## 3.1 Interprétabilité intrinsèque

**Régression logistique.** Un modèle de régression logistique est entraîné sur les données SMOTE et standardisées. Les **coefficients** du modèle donnent une interprétation directe : chaque coefficient mesure l'effet d'une augmentation d'une unité de la variable (après standardisation) sur la tendance du modèle à prédire la classe CKD. Un coefficient positif indique un effet favorable au diagnostic CKD, un coefficient négatif un effet « protecteur ». La figure 3.1 présente ces coefficients triés par valeur absolue. D'après le notebook, les variables à fort effet positif (risque CKD) incluent notamment l'hypertension, le diabète, l'appétit, l'hémoglobine et la gravité spécifique ; à l'inverse, certaines variables (ex. certaines mesures de cellules ou d'œdème) ont un effet négatif.

```
lr = LogisticRegression(max_iter=2000, random_state=RANDOM_STATE,
    class_weight="balanced")
lr.fit(X_train_scaled, y_train_smote)
y_pred_lr = lr.predict(X_test_scaled)
```

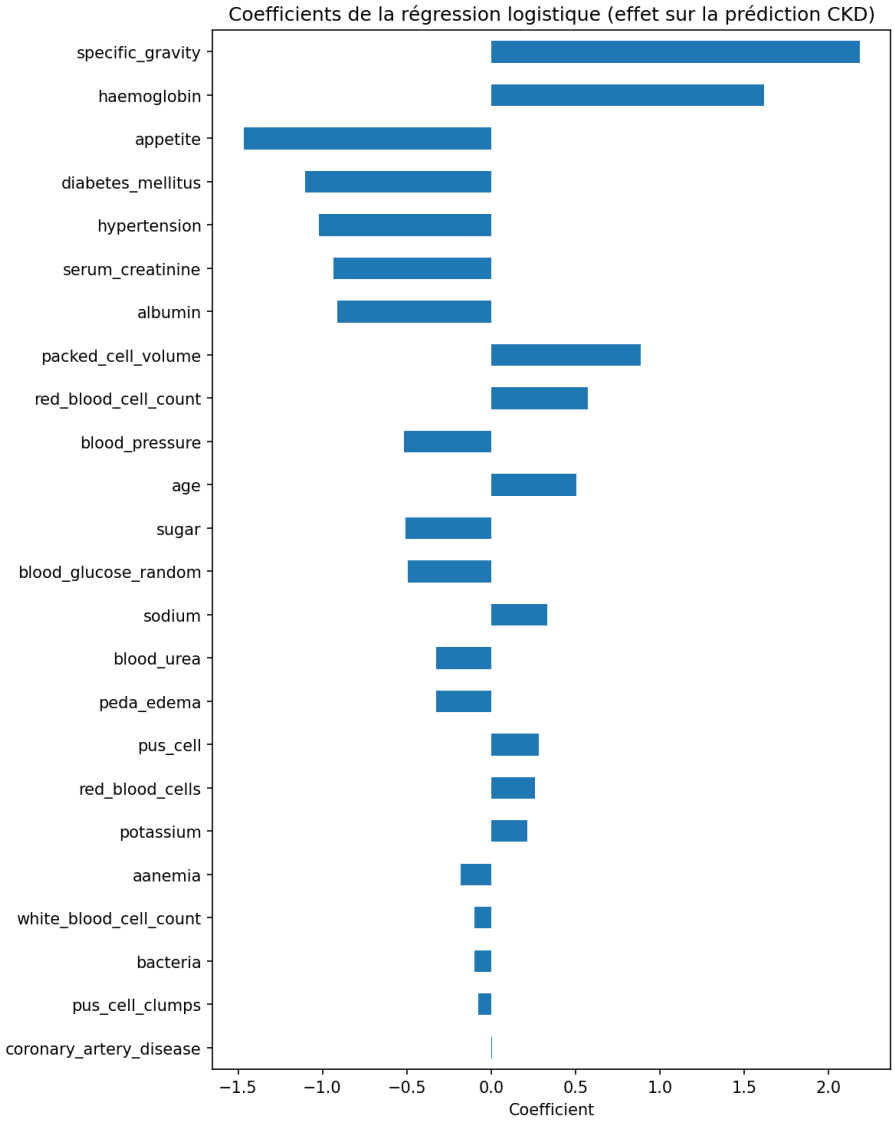


FIGURE 3.1 – Coefficients de la régression logistique (effet sur la prédiction CKD).

**Arbre de décision.** Un arbre de décision est entraîné avec une profondeur maximale de 4 (`max_depth=4`) pour rester lisible et éviter le surapprentissage. Chaque chemin de la racine vers une feuille se lit comme une règle « SI ... ALORS » : par exemple, « SI *specific_ gravity* $\leq$ seuil ET *haemoglobin* $\leq$ seuil ALORS forte probabilité CKD ». La figure 3.2 montre la structure complète de l'arbre (nœuds remplis, étiquettes de classes `ckd` / `not ckd`). L'importance des variables (basée sur l'impureté) met en avant, entre autres, *specific_ gravity*, *haemoglobin* et l'albumine, cohérent avec la littérature sur les marqueurs rénaux.

```
dt = DecisionTreeClassifier(max_depth=4, random_state=RANDOM_STATE)
dt.fit(X_train_scaled, y_train_smote)
y_pred_dt = dt.predict(X_test_scaled)
```

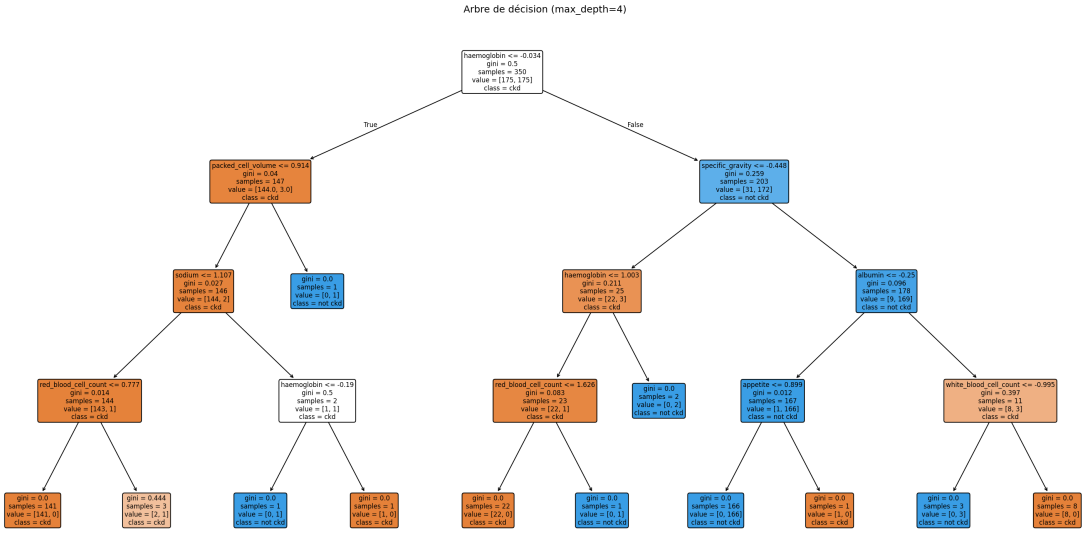


FIGURE 3.2 – Arbre de décision (max_depth=4) pour la prédiction CKD.

## 3.2 Explicabilité post-hoc des modèles opaques

Un **SVM à noyau RBF** est utilisé comme modèle « boîte noire » : il n'offre pas d'interprétation directe des prédictions. L'option `probability=True` est activée pour exposer `predict_proba`, nécessaire à LIME et SHAP. Les performances (exactitude, AUC, matrice de confusion) sont évaluées sur l'ensemble de test ; les valeurs exactes sont reportées dans le notebook. Le modèle sert ensuite de cible pour les explicateurs locaux et globaux.

```
svm = SVC(kernel="rbf", probability=True, random_state=RANDOM_STATE)
svm.fit(X_train_scaled, y_train_smote)
y_pred_svm = svm.predict(X_test_scaled)
```

**LIME (explications locales interprétables, agnostiques au modèle).** LIME approxime localement le comportement du SVM par un modèle interprétable (ex. régression linéaire) autour d'une observation donnée. On perturbe cette observation, on récupère les prédictions du SVM, puis on pondère les points par leur proximité et on ajuste un modèle linéaire dont les coefficients sont les « poids LIME » : ils indiquent la contribution de chaque variable à la prédiction pour cette observation. La figure 3.3 montre ces poids pour une observation de test (patient) ; les barres vertes (positives) et rouges (négatives) indiquent quelles variables poussent vers ou contre la classe CKD. Une limite connue de LIME est l'instabilité des explications d'une exécution à l'autre (variabilité de l'échantillonnage).

```
def predict_proba_ckd(X_arr):
    return svm.predict_proba(X_arr)[:, 0]
explainer_lime = lime.lime_tabular.LimeTabularExplainer(
    X_train_scaled.values, feature_names=feature_names,
    class_names=["ckd", "not␣ckd"], mode="regression", random_state=
        RANDOM_STATE)
exp = explainer_lime.explain_instance(instance, predict_proba_ckd,
   num_features=10)
```

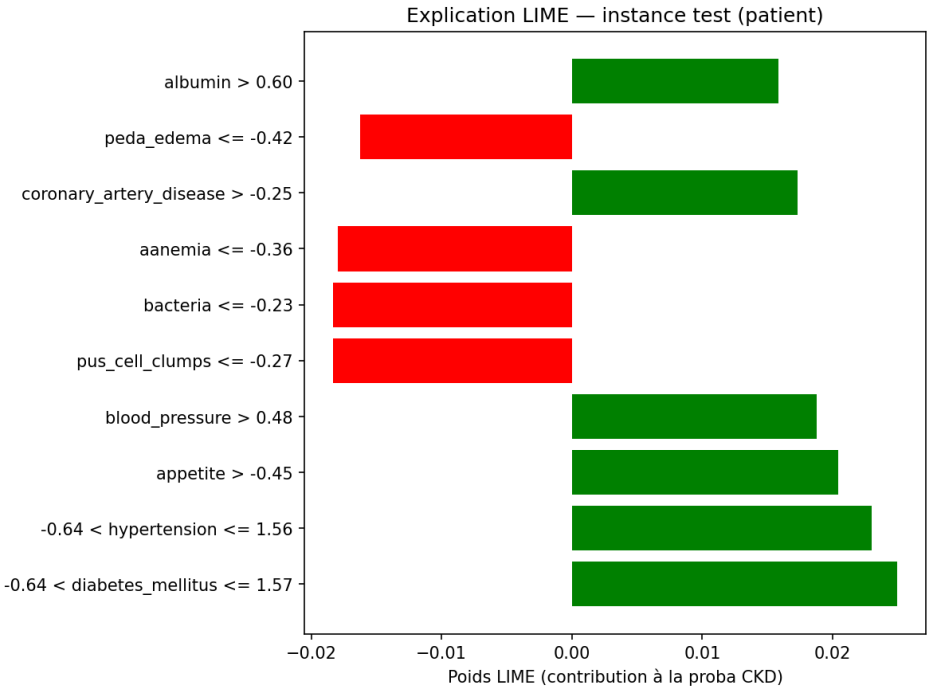


FIGURE 3.3 – Poids LIME pour une observation de test : contribution de chaque variable à la probabilité CKD.

**SHAP (explications additives de Shapley).** SHAP repose sur les **valeurs de Shapley** de la théorie des jeux : chaque variable reçoit une contribution (valeur SHAP) à la prédiction, avec la propriété d'additivité (somme des SHAP = écart entre la prédiction et la moyenne). Un explicateur à noyau (**KernelExplainer**) est utilisé avec un jeu de fond (échantillon d'entraînement) et une fonction de prédiction donnant la probabilité de la classe CKD. La figure 3.4 présente les valeurs SHAP pour la même observation que LIME (explication locale) ; la figure 3.5 est un graphique récapitulatif global sur un échantillon du jeu de test : chaque point correspond à une observation, la couleur indique la valeur de la variable et l'axe horizontal la valeur SHAP. On peut ainsi identifier les variables les plus influentes globalement et le sens de leur effet (hautes valeurs en rouge, basses en bleu). Les explications SHAP sont en général plus stables que LIME et offrent une vue à la fois locale (par observation) et globale (résumé sur l'échantillon).

```
def predict_proba_ckd_shap(X):
    return svm.predict_proba(X)[:, 0]
background = X_train_scaled.sample(50, random_state=RANDOM_STATE)
explainer_shap = shap.KernelExplainer(predict_proba_ckd_shap, background
    , link="logit")
shap_values_one = explainer_shap.shap_values(instance, nsamples=100)
```

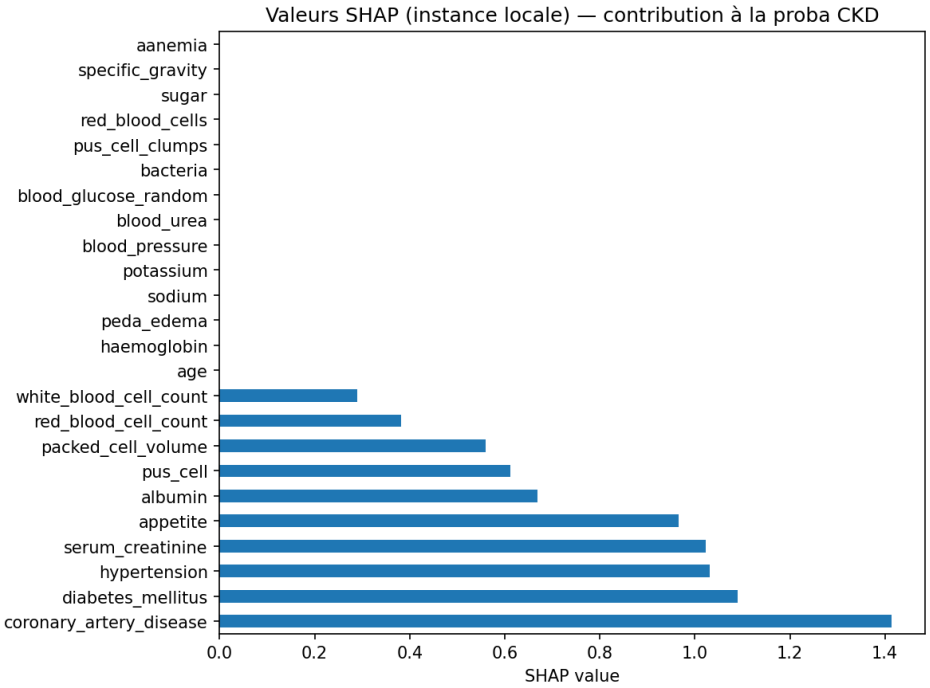


FIGURE 3.4 – Valeurs SHAP (explication locale) pour une observation de test.

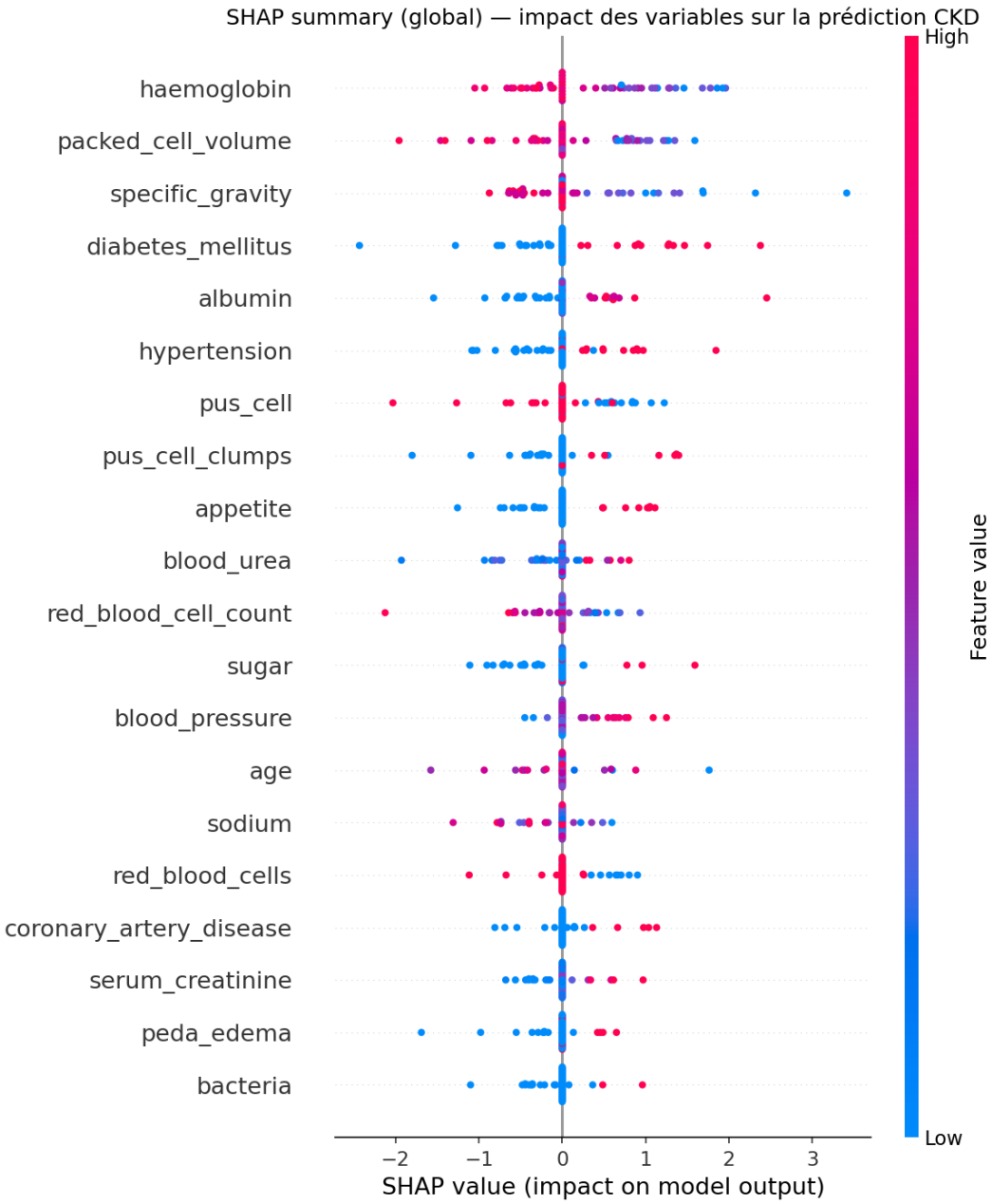


FIGURE 3.5 – Résumé SHAP (vue globale) : impact des variables sur la prédiction CKD.

## 3.3 Synthèse

Les modèles intrinsèquement interprétables (régression logistique, arbre de décision) fournissent directement des règles et des coefficients exploitables pour comprendre les prédictions. Pour un modèle opaque comme le SVM, les méthodes post-hoc LIME et SHAP permettent d'expliquer des prédictions individuelles (LIME, SHAP local) et de résumer l'importance des variables à l'échelle du jeu (SHAP global). La combinaison des deux approches (modèles interprétables pour la transparence et méthodes post-hoc pour les modèles complexes) couvre un large besoin en explicabilité dans le cadre de la prédiction de la maladie rénale chronique.

# 4 Explicabilité des Modèles de Combinaison (Bagging et Boosting)

# 5 Approches Avancées d'Explicabilité